

Transformer From Scratch

Maximillian Roach

August 2026

1 Problems

Problem 1 (unicode1). Questions follow:

(a) What Unicode character does `char(0)` return?

This corresponds to the null character.

(b) How does the character's string representation (`__repr__()`) differ from its printed representation. The string representation is just nothing while the other representation is `'\x00'`.

(c) When you don't use `print`, the `repr` representation is used, but when you use `print` the null character representation is given.

Problem 2 (unicode2). Questions follows

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encoding for various input strings.

UTF-32 is probably the least appealing it guarantees 4 bytes (32 bits). It is size that it has this fixed size, but then it also takes up more space. For a given string, the utf-8 encoding has the least length, while the utf-32 encoding has the greatest length. When doing language modeling, we probably prefer shorter sequences of text for training and inference since that allows for nicer gradients.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

This function takes the string of bytes from the utf-8 encoding of a string and tries to decode it into the original string. The issue is that it processes byte one at a time. For example, it takes byte `i` and decodes it with the standard utf-8 decoder. The issue is that utf-8 encoding doesn't assign a single byte to each character; it uses multiple bytes to capture some characters. All ASCII characters can be captured with one byte, so this assumes that the original string contains only ASCII.

(c) Give a two-byte sequence that does decode to any Unicode character(s). When decoding to unicode, the leading byte tells us how many bytes follow for the given character. Additionally, each following character must begin with 10 (when we convert it binary). Consider `E4 41`, where each is a byte in hexadecimal form. `E4` says that 2 bytes follow, but only one byte follows and does have the correct prefix.

Problem 3 (Train BPE). I'll discuss some of the things that I learned while doing this problem

1. Before tokenizing the text, we need to pretokenize it. Pretokenization is essential because otherwise every time we need to compute pair frequencies we must go through the entire document. This is extremely inefficient. To pretokenize, we break up the text into items like words (though not exactly), returning the frequency of each word in the vocabulary. This is useful because then we can iterate through each word in the vocabulary and compute the frequency of all of its pairs. But to calculate the frequency of a pair, we can add the frequency of the word. Realize how this saves us so much time because otherwise we would be just incrementing each pair by 1 each time we find one.

2. The pretokenization stage can be parallelized somewhat. The first thing we want to do is separate the text file into chunks. We will have one chunk for each work we will use for parallelizing. For this problem, we were given a function to determine the chunk boundaries. To determine the chunk boundaries, we just separate on a certain special token. I was confused why they gave a list of special tokens even though we separate on a single special token. The reason for this is that we just need some special token to separate the document, it's not really important what it is. It just ensures that we aren't splitting in the middle of a document. Then we half each chunk to a worker and that worker processes the chunk and returns a counter that has the count for each word in the vocabulary for that chunk. After the parallel process is done, we merge all the counters together to get one overall vocabulary count. It gets a little confusing when we convert between bytes and strings. I believe that the chunks are broken up as bytes and then in the worker function we convert the chunk into strings. Also note that we further separate the chunk into documents based on the the special tokens.
3. I also learned a little bit about reading files. When you use `f.read()` that reads the entire file. But there is actually a pointer that points to where the file will begin reading. The function that determines the chunk boundaries returns a list of these chunk boundaries. Then we zip them in a certain way so that we get a little of tuples of these boundaries. Then we want to pass each of these boundary pairs to a worker to process that chunk. Then `f.seek(start)` sets the file pointer at start. Then we call `f.read(end-start)` to get the chunk with start and end as the endpoints. This works because `f.read(a)` reads a bytes ahead of the pointer.
4. While processing a single chunk, we break it up into documents and then we process each document separately. But all of the frequencies go in the same counter. At this step, we use the GPT tokenizer regex pattern to divide the document into word-like tokens.
5. This is probably more of a personal implementation issue, but in practice the get-stats and merge-words functions (you can look at the original paper if you don't remember) use tuples as representations of the words. In their naive example they used space-separated strings, but really we want to use tuples. But then it's confusing because when pretokenizing, we process the words as strings, so then I needed a helper function to convert the vocab dictionary from pretokenization to a vocab dictionary with tuples as keys.
6. The merge-word function is more complicated in practice than in the naive example. In that example they just found the most common pair and then went through each word and substituted it in. Then they updated that new word's frequency to the original frequency. Then they do the whole thing again by recalculating the pair frequencies. This works well for simple examples, but we don't want to recompute pair frequencies every time we do a merge. Observe that the only frequencies that change are the containing one of the best pair elements. Therefore, we only need to update the frequencies of these pairs. the character to the left of a replaced pair used to be paired with the left element of the new pair. Now we decrease that frequency by the frequency of the word. Note that we don't set it to 0 since that pair may occur in other places. We do the same for the pair corresponding to the right character of the pair and the character to the right of the replacement. However, we do set the frequency of the pair we replaced to 0 since we replaced every occurrence of that pair and it must have frequency 0 now. Therefore, when we word-merge, we just have to adjust the frequencies that are in the immediate vicinity of the pair we replaced. Also note that before when we updated the new word into the new dictionary, we just set it equal to the old word's frequency. We don't want to do this anymore because with a very large vocabulary, you may have multiple words that converge onto a single word, so you should add to a running count (just use get with 0 as a default).
7. In the overall train-bpe function, you want to return a list corresponding to the vocabulary that has all the tokens, the original 256 tokens corresponding to all the bytes, and then also the byte versions of the special tokens. Initially I forgot to return the original characters, so remember that when we are doing word-merges, we aren't eliminating the vocabulary. Instead we are making it more efficient by adding common sequences.

Problem 4 (Train BPE on TinyStories). (a) It took 47 seconds. The longest word in the vocabulary is "accomplishment". The peak memory usage is 46 MB. It talks about getting under two minutes when you use the hint, but I think I already implemented those things so that's why mine is so fast.

(b) By far the most time is spent on finding the length of things. After that merging words and the vocabulary take a lot of time, but it's really the len function.

Problem 5 (Train BPE on OpenWebText). (a) my computer broke.

Problem 6 (Tokenizer). I'll talk about some things I learned from this problem.

1. During tokenization we may be passed some special tokens. Then we have to add these special tokens to the vocabulary so we can split and process them during pretokenize.
2. The from-files functions needs a class method decorator so that the interpreter knows it's a class function.
3. I broke up the encode function in an encode-word function and the full encode function. In the encode word function, we take a word (extracting using the GPT regex pattern). Then we want to keep finding pairs in the word that can be merged. We do this by looking at the pairs in the merge list. However, the order in which we merge is very important. We want to merge the pairs that are earliest in the merge list because those are the ones that we formed earliest in the training process. Because of this, we scan the whole word and first all the pairs. From these pairs we get the ones that correspond to merges. Then we find the pair that occurs earliest in the merge list. This becomes the pair that we'll use for replacement. We then replace that pair in the word to form the new word. We do this until our word has no pairs in the merge list. We then take the word and convert each byte sequence (corresponding to some token) to a token id using our vocabulary dictionary. Therefore, in the end our word is a sequence of tokens.
4. To encode a full text string, we first split by the special tokens. However, we need to do this in a way so that we still know where the special tokens are because we want to put their token ids into the encoded string in place. To do this, we alter the regex pattern slightly by placing parentheses around it. This tells the regex to still include the special tokens that we split by. After this, we have split the string into documents but also have the special tokens in between the documents. if we that a document is a special token, we just append that special token's id to the encoding list. Otherwise, we encode the document with the encode word function. Therefore we we get the encoding of each document with the proper token ids for the special tokens in between the documents.
5. The encode-iterable function allows us to process a large file one line at a time. For each line we want to encode it, then we want to return the token ids one id at a time.
6. The decode function is pretty simple. We are passed a sequence of token ids. We first convert the token ids into byte sequences. Then we concatenate all these bytes together and decode that long byte sequence to get the original string.

Problem 7 (Linear). In this problem we implement a simple linear layer.

1. nn.Parameter allows us to pass some tensor and then it marks those values as being parameters, so it tracks their gradients
2. You have to call the super of the nn.Module class when you inherit from it. This sets up a lot of important stuff like tracking parameters.
3. I'm a bit confused about how the inputs are being passed, but I think so that we use standard linear algebra notation, the input is being passed so that's dimension comes last and all the batch dimensions come first.

4. trunc-normal allows us to initialize weights but you call it after already allocating space for those weights.

Problem 8 (Embedding). This is pretty much the same as the linear. Instead of doing matrix multiplication we index into the embedding matrix.

Problem 9 (RMSNorm). RMSNorm layer normalization is used before transformer blocks to normalize activations.

1. You can remember what RMS does by going outside to inside. It stands for Root Mean Square. On the outside we take a square root. Inside that we are taking a mean. Then inside that we are squaring. The general process is that for some set of activations, we take the mean of the squares of those activations, then take the square root of that.
2. When we initialize the learnable g weights, they should have shape (d-model,) instead of (d-model, 1). If you do it the second way then broadcasting can't happen. This is because broadcasting aligns on the right, so you will try to move (d-model,1) to the right but that 1 dimension won't match the dimension of the input, only d-model will. Then because d-model matches on the right, the other dimensions will fill in and allow broadcasting to happen.
3. Because we are squaring the weights, we have to be careful about overflowing. To prevent this, we cast the input to float32, perform the calculations to get the result, then cast it back to its original data type.
4. The shape of the output should be the same as the shape of the input. All we are doing to the input is dividing each entry by a constant (which is the RMS of its set of activations) then multiplying each entry by a constant.

Problem 10 (Positionwise Feedforward). I implemented SwiGLU for the feedforward layer.

1. The feedforward network in the original transformer paper used ReLU. However, now something called SwiGLU is used. This is used as the entire feedforward network in a transformer block.
2. The feedforward network is positionwise, which means that it's only applied to a single token at a time. The token representations don't interact with each. The reason for this is that in attention layers the tokens interacted with each other, so now we want some processing of just each token alone.
3. SiLU is a type of nonlinearity where you multiply x by the sigmoid of x . This is very similar in shape to ReLU except that it's smooth at 0.
4. A GLU (gated linear unit) is the component-wise multiplication between a linear transformation of the input and a sigmoid of a linear transformation of the input. The intuition behind this is that we can fight the vanishing gradient problem by having a linear transformation of the input, but then we can also get a nonlinear with the sigmoid of a linear transformation of the input.
5. The SiLU and GLU are put together by taking the component-wise multiplication between SiLU of a linear transform with another linear transform (this constitutes the GLU), then taking a linear transform of that.

Problem 11 (Rope). What is learned about positional embeddings.

1. RoPE stands for rotary positional embeddings. Because attention doesn't directly encode for the positions of tokens, we need to inject positional information into the vectors we supply to attention. The standard approach for doing this is sinusoidal positional embeddings. This assigns a unique vector to each token in a sequence. For each position in a sequence, it uses that token's position along with the dimension position to assign a value in the vector. If it is an even dimension then sine is used and if it's an odd dimension then cosine is used. The intuition behind this is that it's like

each dimension corresponds to an arm on an analog clock. A smaller dimension results in a shorter frequency based on the formula for determining the vector values. A larger dimension results in a larger frequency. Then as we go through each position, each arm is moving at a different speed. The arms with shorter frequencies (corresponding to earlier dimensions) will reach the same position (relative to sine or cosine's range) multiple times. However, this is okay because just like that happens for a clock, each combination of the arms corresponds to a unique position (roughly).

2. Sinusoidal positional embeddings are performed before we supply the tokens to the attention mechanism. We have the token representation and add the positional embedding and then supply that vector to attention. In theory the sine and cosine functions should incorporate information about relative distance since if we add some distance k to a positional embedding, that is the same as multiplying it by applying a linear transformation to it that only relies on that distance. However, because we are applying the positional embedding before attention, these relationships get scrambled and are hard to interpret for attention.
3. The fix is that we instead apply positional embeddings after the key and query vectors are computed. The general idea is that we take a query/key vector and divide it into adjacent pairs. We take each pair and rotate by some angle that is dependent on that pair's position in the vector and that token (vector) position in the sequence. However, because we are doing this after getting the queries and keys, the rotational/positional relationships are not scrambled and instead directly on the surface.
4. The matrix multiplication is pretty complicated for this one but the general idea is that we first get the pairs from the x tensor by using all the first dimensions, then taking the last dimension and turning it into $d\text{-model} / 2$ by 2, so that we have 2 columns. This divides all the terms in token into pairs and arranges them correctly. Then we can index into this pairs tensor and extract the x_0 terms and x_1 terms. I thought at this point we would have to do some complicated matrix multiplication, but observe that we can just look at each component of the product that we want since there are only two terms in each product (roughly). $\text{res}_0 = \cosine * x_0 - \text{sine} * x_1$ and $\text{res}_1 = \text{sine} * x_0 + \cosine * x_1$, where the sine and cosine matrices are obtained by indexing the proper token positions from a precomputed cosine and sine matrices (we have the dimension of the tokens and the max sequence length so we can recompute all the cosines and sines and not have to recompute them every time we want to apply rope to a new vector. This calculations are done at init).
5. After getting the two components of the result, we want to combine them back into a tensor of the same shape as x . Each has dimension $(..., d\text{-model} // 2)$. Then we want to combine along a final dimension so we can get back that 2 in the last dimension (because this should have the same dimension the x pairs tensor, as there is one new vector of the same dimension for every pair). Rather than using `torch.concat`, we use `torch.stack`. `torch.concat` is used when you combine two vectors into the same dimension. For example, calling `concat` on $[1,2,3]$ and $[4,5,6]$ with `dim=1` would give $[1,2,3,4,6]$. But calling `torch.stack` with dimension 0 would result in $[[1,2,3], [4,5,6]]$, like we're actually stacking the matrices on top of each other. You should think of `concat` as appending an existing tensor onto another tensor, keeping the dimensions the same, but `torch.stack` adds a new dimension. To see why we want a new dimension, just consider the two res_0 and res_1 vectors as vectors of dimension $d\text{-model} // 2$. We need to combine them so each row has the res_0 term and the res_1 term. If you concatenate them you just append res_0 onto the end of res_1 and though you get the right dimension as the original x , the order is completely wrong. Then after stacking we reshape back to the original x .

Problem 12 (Softmax). Implementing the softmax function along a given dimension.

1. I was very confused about how we're supposed to implement softmax. Specifically, I didn't know what the given dimension means. The fact that we're performing softmax over a specific dimension shouldn't confuse you. Just think of it as performing softmax over the last dimension. In our output we have a bunch of values and from those values we want to get a softmax, and then there's a softmax distribution for every combination of the other dimensions which may correspond to batches or sequence lengths.

2. It's really important to pass `keepdim=True` to the `amax` and `sum` functions. Without this, it will eliminate that dimension from the resulting vector. For example, if you have a matrix of shape `[2, 3]` and you perform a sum over `dim=1`, that takes the sum over the rows. Then the shape reduces to `[2]`. However, because broadcasting is done by first aligning to the right, we will slide this 2 over to the right but it almost certainly doesn't match the dimension on the right of what we were planning to use it for. By passing `keepdim=True`, the dimension that we reduce over will stay 1 so that broadcasting works. Even if a matrix reduces to shape `[a,]`, the `a` will be shifted to the right.
3. Even though the softmax outputs probabilities between 0 and 1, it may blow up to `inf/inf` when we have large values. To fix this, we subtract the max value in the vector from all the values so that the maximum value is 0.

Problem 13 (Scaled Dot Product Attention). I implement the standard attention formula.

1. The way you should think about attention is through the queries `Q`, keys `K`, and values `V`. Think of it like a dictionary. We have key-value pairs and we want to make some query. For each key, there is a corresponding value (they have the same row dimension but column dimension is different because those represent embeddings). Then for a single query, we compare it against every key. We softmax this to get the importance of each key to the query. If we have `n` queries and `m` keys/values, the result after softmaxing is an `n x m` matrix. We multiply this by an `m x dv` matrix, where `dv` is (I think) a hyperparameter that determines the dimension of the value embeddings. So a given row of the `n x m` matrix tells us how important each key is. We then multiply by the value matrix. For row 1, we get the sum of each key's attention weight multiplied by its corresponding value. Therefore, the output returned for a given query is the product of all the values weighted by how important each key is to the query.
2. It helps if you interpret `softmax(qkT)` as attention weights (though remember to divide by the square root of the embedding dimension). Each row of this matrix corresponds to a given query. Each entry corresponds to the importance of that key to the query. This occurs because we take the dot product of that query with every key and that builds out the row. Then we take this matrix and multiply it by the values. This then weights every value by the attention weight from its key and the query. The output of attention is then an assessment of all the values informed by each key's relationship to the query.
3. A mask can be applied before the softmax. We want to do this so that a query will only attend to some keys. The mask has the same dimension as this matrix, `n` by `m`. If the mask is true at position `(i, j)`, that tells us that query `i` should attend to key `j`. If it's false then we don't want it to attend. This can be a little confusing because this is saying that if it's true then we DON'T want to apply the mask and if it's false, we DO want to apply the mask. But the first way is like saying true means we want the attention between the pair and false is saying that we don't want that attention. Then to actually apply the mask, you find the points where the mask is false and you set those values to `-inf`. Then when these values go through the softmax they are pretty much ignored and go to 0.

Problem 14 (Multihead Self-attention). This was a tough one and I need a lot of help. You should definitely review this.

1. Multihead self-attention performs attention on the input one time for each head. Each head had a slice of a larger weights matrix. The learnable weights are `wQ`, `wK`, `wV`, and `wO`. `wO` is a matrix applied after performing multihead attention to finish the process for self attention.
2. The important thing to realize is that we can still store all the weights in a single weight matrix for each of `wQ`, `wK`, and `wV`. We first want to compute `Q`, `K`, and `V` by matrix multiplying each with the input `X`. Then we take each of these and rearrange them so that `h` is the batch dimension. We perform the rearrangement `... n (h dh) -j ... h n dh`. Observe how move `h` ahead so that when we perform attention with `Q`, `K`, and `V` the attention is performed per head.

3. It's hard to wrap my head around how batch dimensions work and how we can get the separated head attention calculations by just rearranging. Something that helps me is that when matrix multiplication is performed, the last two dimensions are the ones really performing multiplication, and then all the other dimensions before that just determine the number of pairs. In the same way here, because we defined the standard attention to only care about the last two dimensions, we can think about attention being performed on just the last two dimensions and then it's performed one time for each of the batch dimension combinations, which means it's performed separately for each head since we moved the head dimension out of the last two dimensions.
4. So after performing the initial calculations to get Q, K, and V, we rearrange them to move the head dimension to act like a batch dimension, which will allow us to perform self attention per head.
5. It's also quite difficult to determine how we should build the mask. For the mask, we want it so token i only sees the tokens before it. The first thing that's difficult is to know if the query vectors correspond to the rows or the columns. If you look at the standard attention implementation, `qkT` is `n` by `m`, where `n` is the number of queries. Therefore, we know the queries go along the rows. This means that we want the mask to be triangular such that the True values are below the main diagonal and the false values are above the main diagonal. Then if you look at the first row, it will only see the first token (itself) and all the other tokens are False. This type of matrix can be obtained with `torch.tril` (all values above the main diagonal are 0). Because the mask is supposed to be boolean, you can pass `dtype=torch.bool`.
6. After creating the mask we apply the standard dot product attention and because we have rearranged Q, K, V correctly, it will perform attention correctly by separate heads. After this we get some output, but this output still has dimension that corresponds to having that head dimension on the outside. The dimension of the output from each head is `n dh` since `dh` is the slice size of the head. Then we perform the rearrangement `... h n dh -> n (h dh)` so that we're combining `h` and `dh` back into the correct dimension of the original `x`. I believe this is like concatenating the heads together.
7. They also ask that we do multihead self-attention with RoPE. Remember that RoPE is applied to Q, K, V. However, we want the same RoPE to be applied to each head. We don't want to apply it before the rearrangement since then the rotation won't account for how we're splitting by heads afterwards and within those heads the positions won't be encoded correctly.
8. One of the key takeaways is that even though MHSA requires breaking up the original weight matrices so that we have one for each head, we don't have to do this iteratively. We can rearrange and treat the head dimension as a batch dimension so that our standard attention can be applied over the rearranged Q, K, V.

Problem 15 (Transformer Block). I put the previous pieces together to form a transformer block. The architecture is first RMSNorm the input, supply that to the MHSA. Add the input to this result as a residual connection. This take this RMSNorm it and give it to the SwiGLU fn. Then add the output before the SwiGLU to this and return.

1. At first I thought that you're supplying the same residual connection both parts, but you're only supplying the original `x` to the first residual connection. For the second one (the one that goes at the end) you are supplying the output from the attention layer.
2. We are given weights for all of the layers so to efficiently load them and also be able to access them from the transformer block class, we create a load-weights function, then inside that function we call `load-state-dict` for each of the layers.

Problem 16 (Transformer LM). I put transformer blocks together with an RMSNorm then a linear layer.

1. To create layers you need to use `nn.ModuleList` then do list comprehension. Otherwise, the layers you make won't be stored as parameters.

2. To load weights we can include a prefix in the load weights of the transformer block. Then we can pass down the proper prefix and state dict so that the transformer block will know how to load weights into its layer.

Problem 17 (Transformer Accounting). For an $m \times n$ matrix A and an $n \times p$ matrix B, the matrix-matrix product AB requires $2mnp$ FLOPs. To see this, observe that to compute a single entry (i,j) in the matrix AB, there are n multiplications and (technically $n - 1$) n additions. so each entry costs $2n$ FLOPs. Then AB has dimensions $m \times p$, so we multiply this by mp to get $2mnp$ FLOPs.

float32 is greater if you want a lot of precision and a large range. It has 8 bits for exponents and 23 bits for precisions. However, this isn't always necessary, especially for deep learning tasks. An alternative that saves on memory is float16, which has 5 bits for exponents and 10 bits for precision. Each allocates 1 to sign.

A floating point operation (FLOP) is either one addition or one multiplication.

Another way to think about the FLOPs calculation above is that the number of FLOPs for matrix multiplication is equal to $2 (\# \text{ tokens})(\# \text{ parameters})$ since m represents the number of tokens and np represents the number of parameters.

Model FLOPs utilization (MFU) is measure of how well you are using the true capabilities of your hardware. You calculate it by dividing the actual FLOP/s by the promised FLOP/s, where you calculate the first by actually calculating the number of FLOPs your program uses and dividing it by the time it takes the execute the program.

Good MFU is above 0.5. The reason it's around here is that to perform computations you need to transfer the inputs from the memory region to the compute region (accelerator). The computation are then performed in the accelerator and sent back to the memory, so there is time being taken up by transportation between the memory and the accelerator. Therefore, the two things that determine how fast a program runs is it's accelerator speed (FLOP/s) and it's memory bandwidth (bytes/s).

Consider an example. We want to perform relu on some n-dimensional vector. Then the number of FLOPs is clearly n since we are just performing n comparisons. The number of bytes that are used in transportation is dependent on the data type. If we are using bfloat16, then we have 2 bytes per float (since each byte has 8 bits and there are 16 bits). Then for each input, 2 bytes are used to read it (transfer it from memory to compute) and 2 bytes are used to write the output (transfer from compute back to memory). Therefore, the total number of bytes is $4n$. Then we can calculate the communication time by dividing this by the number of bytes per second of your hardware. You could then compare this with the computation time to see which is larger. When performing these calculations we assume that we are able to schedule data transfer and computation perfectly so nothing is sitting idle. Because of this we calculate the total time to be the max of the computation time and the communication time.

The accelerator intensity is the hardware's FLOP/s divided by the hardware's bytes per second. It tells us how much compute is done per byte of data transferred. This uses the numbers provided on a spec sheet, not the actual calculations you compute. The arithmetic intensity tells us for our current workload how much work can we do per byte, where now we use the actual number of FLOPs and bytes for our program.

In terms of the communication and computation times we calculated before, it was clear that if your communication time is greater than your computation time, then you're memory-bound since it's taking more time to do communication, and vice versa. In terms of intensities, you're memory-bound if your arithmetic intensity is less than your accelerator intensity. You're compute-bound if your arithmetic intensity is greater than your accelerator intensity. And actually the algebra is pretty much the same between the two so you can get these relationships by just comparing arithmetic and accelerator intensity than

shifting the terms so that it's like computation time and communication time.

We would rather be compute-bound because that means we are getting the most out of our accelerator rather than just eating away time transferring data. The intuition behind this is that if we aren't memory-bound then we are able to just do computations and maximize the promised computation.

Suppose we want to compute the number of FLOPs needed to perform the forward and backward pass of some layer in a deep linear network. Suppose it has some number layers but we want to zoom in on the second layer. This layer will have input $h1$, weight matrix $w2$, and output matrix $h2$. These are all matrices because you can imagine the input to the first layer as a matrix of shape $B \times D$, where B is the number of batches and D is our embedding dimension or something like that. So all the weight matrices are $D \times D$ and the input and outputs are $B \times D$. The forward pass is pretty simple to compute because we are taking the input matrix $h1$ which is $B \times D$ and multiplying it by a matrix that is $D \times D$. Based on the reasoning we had previously about how to calculate the number of FLOPs for a matrix-matrix multiplication, this comes out to $2 \cdot B \cdot D \cdot D$.

The backward pass is a little more complicated. Observe that the computation we are performing in this layer is $h2 = h1 * w2$, where the operation is multiplication. The job of the backward pass is to compute the gradient of the loss with respect to $w2$ and with respect to $h1$ (it's important that in a layer you also compute the gradient of the loss with respect to the input since that is used in the next layer back). But we are given $dh2$ since that was computed in the backward pass of the layer after this. If you do the calculus, then $dh1 = dh2 * w2$ and $dw2 = dh2 * h1$. In practice you have to worry about transposes but let's not think about that now. Then each of these matrix-matrix multiplication is between a $B \times D$ matrix and a $D \times D$ matrix, so each takes $2 \cdot B \cdot D \cdot D$ FLOPs, so the total FLOPs for the backward pass of this layer is $4 \cdot B \cdot D \cdot D$. Then the total number of FLOPs for the forward and backward pass of this layer is $6 \cdot B \cdot D \cdot D$. If we apply this logic to the entire neural network with more general layer dimensions, the number of FLOPs is $6(\# \text{ data points})(\# \text{ parameters})$.

Problem 18 (Cross-Entropy). Implement cross-entropy loss over batch size and possible sequence length.

1. There are some issues here about when we should keepdim and when we shouldn't. But the first thing to mention is how we can simplify the cross-entropy loss (negative log prob) of a single example. The first thing is that the numerator and denominator of softmax can be separated into a difference of logs. Then the log of the numerator cancels out with the exponent in the numerator so that the term just comes out to the logit of the target. For the denominator, we would like to subtract the max logit from all the logits so that when we exponentiation things won't blow up. To do this, in the denominator expression we can multiply the sum by $\exp(m)$ and $\exp(-m)$, where m is the max logit. Because we can separate the log of a probability into a sum of logs, we get to move $\log(\exp(m))$ outside and it just becomes m . Then we are left with computing the log of the sum of the exponents of shifted logits. I thought you could separate a sum into a product for logarithms but remember that it's the other way around. This simplifies the entire cross-entropy calculation in the log of the exponentiation shifted logits, the max logit, and minus the target logit.
2. We would like to perform the cross-entropy over an entire batch. There are two situations: either we are computing the cross-entropy loss over an entire batch but for logit predictions over some set number of previous tokens, so we have no dimension for the sequence length. Then we want to divide by the batch size to take the average loss. But if we did have sequence length, so that we consider all incremental sequences, then we want to divide by the batch size multiplied by the sequence length. I tried to divide by both but that doesn't generalize to both scenarios, so it's better to just take the mean.
3. The dimension get a little confusing. At least in this implementation, we are covering the first situation described above. This means logits has shape (batch-size, vocab-size) and targets has shape (batch-size,). This is the shape of targets because for each example we have exactly one target and it corresponds to an index of the vocab. I thought that you can just directly index into the logits

with targets, like `logits[targets]`. But this doesn't work because each entry in targets corresponds to an index from the vocabulary. But when we do this fancy indexing, we extract an index that corresponds to a training example, which gives us entire rows of logits corresponding to our target indices. This is definitely not what we want. Instead, we want the vocab word in each example that corresponds to our index. Because of this, we index through each row as the first dimension, and then pass targets to further get the vocab word we want, like `logits[arange(batch-size), targets]`.

4. We want to take the max `m` from each row of logits so that we get the max logit from each example. Then we wish to subtract it from logits. `logits` has shape `(batch-size, vocab-size)`. If we take the max but don't use `keepdim=True`, the max `m` will have shape `(batch-size,)`, so when we broadcast it won't work. Therefore, when computing `m`, we use `keepdim=True` so that it has shape `(batch-size, 1)` and we can broadcast correctly. Consider later that we want to compute `ce = m + 1 - logit-targets`, where `m` is that max, `1` is the log of the sum in the denominator of softmax, and `logit-targets` is the vector containing the target logits. `target-logits` has shape `(batch-size,)` and `1` has the same shape. If we keep `m` as shape `(batch-size, 1)`, then the other will begin broadcasting by shifting to the right and then `m` will try to broadcast over them as `(batch-size, batch-size)`, which is not what we want. In this situation, we want `ce` to be of shape `(batch-size,)` since we want a loss for example. Therefore, we must use `m.squeeze(-1)` so that it becomes shape `(batch-size,)`.

Problem 19 (Learning Rate Tuning). At `1e1`, the loss is pretty bad. At `1e2`, the loss is greater and pretty much goes to 0. At `1e3`, the loss explodes to a very large number.

Problem 20 (AdamW). Writing the AdamW optimizer from scratch. AdamW is a variant of Adam that also incorporates weight decay of the parameters. The point of this decay is to nudge the weights closer to 0 as a regularization technique.

1. AdamW is stateful since we need to keep track of the running first and second moments of each parameter. To update the a weight, we use the weight's running first momentum, running second momentum, and step-adjusted learning rate. I thought this learning rate is stored as a state, but `m` and `v` are the only values that are truly stored as states. You also have the time step `t`. But the step-adjusted learning rate is calculated from `lr` each time and then multiplied by a factor dependent on the time step. The first thing that happens is that you apply the weight decay to weight. This is done using the base learning rate and the weight decay hyperparameter. You don't use the step-adjusted learning rate. Then you get `m` and `v` and update them based on the gradient (first moment) and gradient squared (second moment). Then you update the weight parameter with all of these adjusted values. At the end, store `t`, `v`, and `m` in the state since their running values.
2. We can use `get` with a default value to account for the first iteration of the optimizer. However, for `m` and `v` I thought they would just be initialized to 0. But `m` and `v` are in the same shape as the current parameter `p`. I kept think that `p` is just a scalar, but it could be a whole matrix for a layer of weights. Because of this, `m` and `v` should be initialized to be in 0s in the same shape as `p`.
3. We have hyperparameters that are passed to AdamW when we initialize it. In the step function, we iterate through the groups in `self.param-groups`. The reason we have groups is because you may have different parameter sets for each part of your model. Then you want to iterate through the groups to go over all the parameters. Then in each group you have may have some hyperparameters, so for each group we extract the appropriate hyperparameters, and in this case we're extracting the hyperparameters of AdamW.

Problem 21 (Learning Rate Schedule). The cosine annealing learning rate schedule have a warm-up phase, the actual cosine phase, and a post-annealing phase. The schedule function is given the current iteration step, the min learning rate, max learning rate, the iteration that ends the warm-up phase, and the iteration that ends the cosine-annealing phase. In the warm-up phase, the learning rate increases linearly until it reaches the max learning rate at the end of the warm-up phase. Then the cosine phase starts. I don't understand this phase completely, but just looking at graphs of the learning schedule I see that

it starts at the max learning rate and then goes down to the min learning rate. This constitutes going through one half period of the cosine wave. It goes from $\cos = 1$ to $\cos = 0$. Then after the cosine steps, if you want to keep training the learning rate is the min learning rate.

Problem 22 (Gradient Clipping). We fear that after a backward pass our gradients will be too large, as there may be a spike in some gradient. One approach to fixing this is that you have some maximum that a gradient can be for a weight. If that gradient exceeds some pre-defined limit, then you clamp it to the maximum/minimum. However, this doesn't work completely well because some gradients are being clamped while others aren't making it so the components of the overall gradient isn't changing by the same factor in every direction, resulting in a clamped direction that may not be in the exact same direction as the original gradient. Because this gradient is defined as going in the best direction for minimizing the loss, we would like to maintain that direction while also accounting for exploding gradients. To do this, we define some max l2 norm. Then we take the l2 norm of every parameter's gradients. This is like putting all of the gradients into a single vector then computing the l2 norm of that vector. If the l2 norm exceeds our max l2 norm, then we multiply all the gradients by $\text{max} / (\text{norm} + \text{eps})$ where eps is some small value to prevent division by 0. By dividing by the norm, the norm of the gradient vector becomes 1, then we multiply by M so that the norm of the gradient is our pre-defined max. Because we are scaling every gradient, they all scale in the same direction and overall gradient vector points in the same direction, but we still benefit from clipping the gradient.

Problem 23 (Data loading). A data loader takes a token sequence and returns some number of input/target pairs.

1. I was really confused about the whole point of the data loader. I thought that you're given a batch of token sequences and then you want slice each token sequence so that you can get the inputs and targets offset by 1. The reality is that when we are training we will turn all of the text (which contains multiple documents) into tokens. Then we concatenate all these documents together as their token representations. The documents are separated by end-of-sequence tokens.
2. So we have this huge long string of text and from it we want to get training examples. To do this, we choose some fixed context length m . This makes it so we take a sequence of m tokens and we want to predict the next token. The nice thing about defining this fixed context length is that it means we don't have to pad our examples since they all have the same length. Our intended output from the data loader is batch-size number of training examples where each example is a sequence of m tokens, and each input sequence is paired with a target sequence that is shifted to the right by 1. For example, if batch-size = 1 and $M = 3$, then the example would be $[[x_1, x_2, x_3], [x_2, x_3, x_4]]$. For example, for index 1 it's saying that we get to see x_1, x_2 and we predict x_3 (which is at index 1 in the targets).
3. To generate the examples, we want to sample random sequence from the single sequence we're given. This is really easy because we can choose our starting index randomly from $[0, \text{len}(\text{seq}) - \text{context-length}]$, where $\text{len}(\text{seq})$ is the length of the token sequence we're given. These are possible indices we can use as the starting point of our sequence. The upper limit is determined by the fact that from the start we must go ahead context-length characters so it's chosen so that we don't go out of bounds of the sequence. Then we sample batch-size of these start indices, and then form the inputs by sampling from the given sequence with those indices. We generate the targets the same way except we shift by one to the right.
4. After generating the array of training examples, we have to stack them on top each other so that the first dimension is batch-size. Because the input sequence is given to us as a numpy array, we have use `np.stack`. Then after stacking the inputs and targets so they're in the right shape, we need to convert both to torch tensors. This can be done with `torch.from_numpy()`.

Problem 24 (Checkpointing). Implement a load-checkpoint and save-checkpoint function.

1. We want a way to save model weights and optimizer parameters (if we're using an optimizer like AdamW that stores running states), and then also a way to load those checkpoints later. For the save-checkpoint function we can use the `state-dict()` method of the model and optimizer and then package these along with the ending iteration into a dictionary. We can then use `torch.save` to save it at a file path.
2. When we want to load the checkpoint, we can use `torch.load` to get the dictionary corresponding to the given path. Then we can extract each of the state dicts and use `load-state-dict` to load the weights into the model and optimizer.

Problem 25 (Training loop). I implement a standard training loop for the LM model.

1. I was a bit confused at how the tokens are loaded. Before we trained the tokenizer on TinyStories so that it determines the vocab and merges from the training data. From here we can supply that training data to the tokenizer with those vocab and merges and it will output the proper token sequences. When we upload the tokens we use `np.memmap` so that if the tokens are held in a very large file there isn't an overload.
2. We can use the `argparse` library to be able to use command line arguments.
3. On each iteration of the training loop, we first sample a batch of training examples. Then we use the training examples to produce logits from the model. Those logits and the targets are given to the cross entropy loss to get our loss. Then gradient clipping is applied (BEFORE the OPTIMIZER STEP) and then the optimizer steps.

Problem 26 (Decode). We handle taking a prompt and producing an output.

1. When we want to generate text, we supply a sequence of tokens corresponding to the prompt. Then we pass that token sequence to the model. At each step we take the last entry of the model's output to get the next token logits. From these logits, we can divide by the temperature for temperature scaling before the softmax.
2. The way that temperature scaling works is that as the temperature approaches 0, the logit with the highest original probability becomes more isolated. That is, if the temperature is 0 then after softmax we will get a one-hot vector with the position corresponding to the highest logit being 1. Then when we sample from the distribution we'll just choose the token corresponding to that logit. This means that the lower the temperature is, the more likely we are to choose the word that is most probable.
3. After softmaxing with temperature scaling, we want to perform top-p sampling. Top-p sampling involves taking some probability threshold. Then we sort the probability distribution and keep taking the top probability. We accumulate this sum until it exceeds the threshold. Then the indices we have collected will determine the new probability distribution. We zero out the other words and then normalize on just these probabilities. The purpose of this is that we take only the tokens that the model is most confident about and then sample from them.

Problem 27 (Layer Norm Ablation). Once I removed layer norm I immediately got NaN losses. This is because layer norm normalizes your data, but once we remove it training becomes extremely unstable since initializing at bad values results in exploding outputs when going through many layers.

Problem 28 (Pre-norm Ablation). When we use post-norm instead of pre-norm, the loss stops decreasing at a much larger point. Rather than going down close to 2 like with pre-norm, using the same parameters as in my best performance model, the loss is around 5.8.